Adyen Staff Engineer (Mobile SDK) — Interview Prep

Oct 30, 2026 · @JP

What Adyen is testing

Adyen is hiring a hands-on technical leader who can design payment SDKs, ship mobile code, and make four distributed teams faster. Every stage below tests the same three muscles from different angles: Technical (design/architecture), Product & Business (evaluate, plan, synthesize), and Mentorship (learn and teach).

The Payments (Mobile & Terminal Experiences) description is the one to prepare against. Its signals are unusually specific, so mirror its language in your answers:

- **Hands-on, not just directing.** They want shipped and owned production mobile code. Expect follow-ups like "what did you personally write?"
- **SDK design.** Merchant- or consumer-facing SDKs, platform thinking, simple services that serve many products.
- **Cross-platform willingness.** Strong in Android (Kotlin) or iOS (Swift), and a credible plan to cross-train on the other.
- **Influence without authority.** Push back with data and POCs, build alignment across time zones.
- **Mentoring by level.** Coach seniors to solve hard problems alone; explain the why to juniors.
- **Operational excellence.** Observability, testing, and work with OS providers and hardware vendors.

Stage	What they are really scoring	You win when
Technical design	Structured thinking, mobile + payments depth, trade-offs, security instincts	You clarify before drawing, design the public API first, and name what you would not build
Integration project	Judgment, pushback, business alignment, cross-team communication	You turn an over-ambitious ask into a phased plan everyone can commit to

Stage	What they are really scoring	You win when
Chat with manager	Clarity, self-awareness, scope, how you plan and scale people	You answer first, then draw your work as a map of customers, systems, teams, and impact

Your angle: you are an Android/Kotlin engineer moving from Views to Compose, and you are building a Kotlin SDK that talks to dive computers and downloads dive logs. That is directly relevant: device connectivity, protocol handling, an SDK API, and a sample app are the same problems as payment terminals and mobile SDKs. Use it as an anchor story in every stage. Your stated gap to address proactively: iOS depth, so bring a concrete cross-training plan (Mentorship section).

Homework before any round: skim Adyen's public Android and iOS SDK repos on GitHub and the online payments docs. Note how they split Drop-in (all-in-one UI) from Components (per payment method), how the Sessions flow works, and how they handle 3D Secure, Google Pay, and Tap to Pay. Product details here are from general knowledge; confirm them on docs.adyen.com before quoting them.

Stage 1: Technical design whiteboard

Run every design prompt through the same six-step frame, and spend the first ten minutes understanding before drawing a single box. Staff candidates fail this round by jumping to architecture; they pass by making the interviewer agree on the problem first.

The six-step frame (60-minute round)

1. **Clarify (5–8 min).** Who integrates this (merchant developers) and who uses it (shoppers, store staff)? Which platforms and minimum OS versions? Which payment methods and regions? What already exists?
2. **Requirements with numbers (5 min).** Functional: what flows must work. Non-functional: binary size budget, cold-start cost, availability, latency to show the payment sheet, PCI scope, accessibility, languages.
3. **Public API first (10 min).** For an SDK the API is the product. Write the 5–10 lines a merchant would type to integrate. Everything else serves that.
4. **Architecture (10 min).** Layers, modules, state, threading, data flow between merchant app, merchant backend, and Adyen.

5. **Deep dives (15 min).** Let the interviewer pick, but be ready on security, failure handling, observability, and release.
6. **Trade-offs and evolution (5 min).** What you chose not to build, what breaks at 10x, and how v2 ships without breaking v1 merchants.

Say out loud at each step: the assumption, the trade-off, and the decision. "I'm assuming X; the trade-off is Y vs Z; I'd pick Y because of the merchant upgrade problem."

Mobile SDK design checklist

The key insight for SDKs: **you do not control upgrades.** A merchant may ship your v4 for three years. That single fact drives most SDK decisions.

- **Small, stable public surface.** Kotlin explicit API mode, a binary-compatibility validator in CI, Swift library evolution. Internal by default.
- **Semantic versioning and a deprecation policy.** Deprecate for at least one major version, with migration guides and lint hints.
- **Modularity for binary size.** A core module plus one module per payment method or capability, so a merchant pulls only what they use.
- **Minimal transitive dependencies.** Every library you depend on becomes the merchant's version conflict.
- **Lifecycle correctness.** Android process death, configuration change, and returning from a 3DS or banking-app redirect. iOS backgrounding and scene changes. The payment state must survive all of these.
- **Threading contract.** Document which thread callbacks arrive on. Offer coroutines/Flow and async/await, plus a Java-friendly and Objective-C-friendly path.
- **Customization without breaking security.** Theming tokens (colors, fonts, corner radius) and text overrides, not arbitrary views inside secure card fields.
- **Accessibility and localization** as requirements, not polish.
- **UI toolkit strategy.** Compose and SwiftUI entry points alongside Views and UIKit, sharing one state engine underneath.

Security deep dive

Security questions in payments start from one goal: **keep the merchant out of PCI scope.**

- **Client-side encryption.** Card data is encrypted in the SDK with Adyen's public key, so the raw card number never touches merchant code or servers.

- **No secrets in the app.** The client key is public by design, scoped and origin-restricted. The API key lives only on the merchant backend.
- **Transport.** TLS everywhere. Treat certificate pinning as a trade-off: it stops interception, but a key rotation can brick every merchant app still shipping an old SDK. If you pin, pin to backup keys and have a remote kill switch.
- **Device integrity.** Play Integrity and App Attest as risk signals, not hard blocks. Required for Tap to Pay style acceptance.
- **Secure storage.** Android Keystore and iOS Keychain; store as little as possible.
- **Logging hygiene.** Never log card data, tokens, or personal data. Enforce with lint rules and tests.
- **Supply chain.** Signed artifacts, dependency scanning, SBOM, reproducible builds, protected release credentials.
- **Threat model out loud.** Name the attacker (malicious app on device, network attacker, compromised merchant app, insider) and the asset (card data, session, device).

Reliability and payment correctness

- **Never double charge.** Idempotency keys on payment calls; retry only operations that are safe to retry; exponential backoff with jitter.
- **Explicit state machine.** Idle, collecting input, submitting, awaiting action (3DS, redirect, QR), result. Every state persisted and resumable.
- **Unknown outcomes.** A timeout is not a failure. Resolve with a status lookup or webhook before telling the shopper anything.
- **Graceful degradation.** If a wallet is unavailable, hide it; do not fail the checkout.
- **Terminals and offline.** Store-and-forward only with risk limits agreed with Risk and Legal; it is a business decision as much as a technical one.

Observability

- Privacy-safe telemetry: funnel events (sheet shown, method selected, submitted, authorised, abandoned), errors, latency.
- Every event tagged with SDK version, platform, and integration type, so you can see version adoption and attribute crashes.
- Remote configuration and kill switches, because you cannot hot-fix code already shipped inside merchant apps.

- Dashboards for the long tail: what share of volume runs on SDK versions you want to retire.

Delivery and CI/CD

- **On every pull request:** ktlint/detekt and SwiftLint, unit tests, screenshot tests (Paparazzi or Roborazzi on Android, snapshot tests on iOS), API compatibility check, binary size diff with a budget.
- **Nightly:** device-farm runs across OS versions and manufacturers, end-to-end payments against the test environment, sample app builds.
- **Release train:** a predictable cadence (for example every two to four weeks), release candidates tested by pilot merchants, changelog and migration notes generated from labels.
- **Distribution:** Maven Central, Swift Package Manager, CocoaPods; signed and verified.
- **Rollback reality:** you cannot roll back an SDK in the field. Your levers are server-side flags, remote config, and fast patch releases. Say this explicitly; it shows Staff-level understanding.

Scalability: systems and teams

- **Traffic.** SDK calls hit the merchant backend, which calls Adyen. Peak events like Black Friday mean retry storms; jitter and client-side rate limits matter.
- **Shared core across platforms.** Options: fully native per platform, a Kotlin Multiplatform shared core with native UI, or Flutter/React Native wrappers over native. The common Staff answer: native UI, shared logic only where it reduces bugs, thin wrappers for cross-platform frameworks.
- **Teams.** Clear module ownership, an RFC process for public API changes, architecture decision records, and a contribution guide so other teams can add a payment method without your team as a bottleneck.

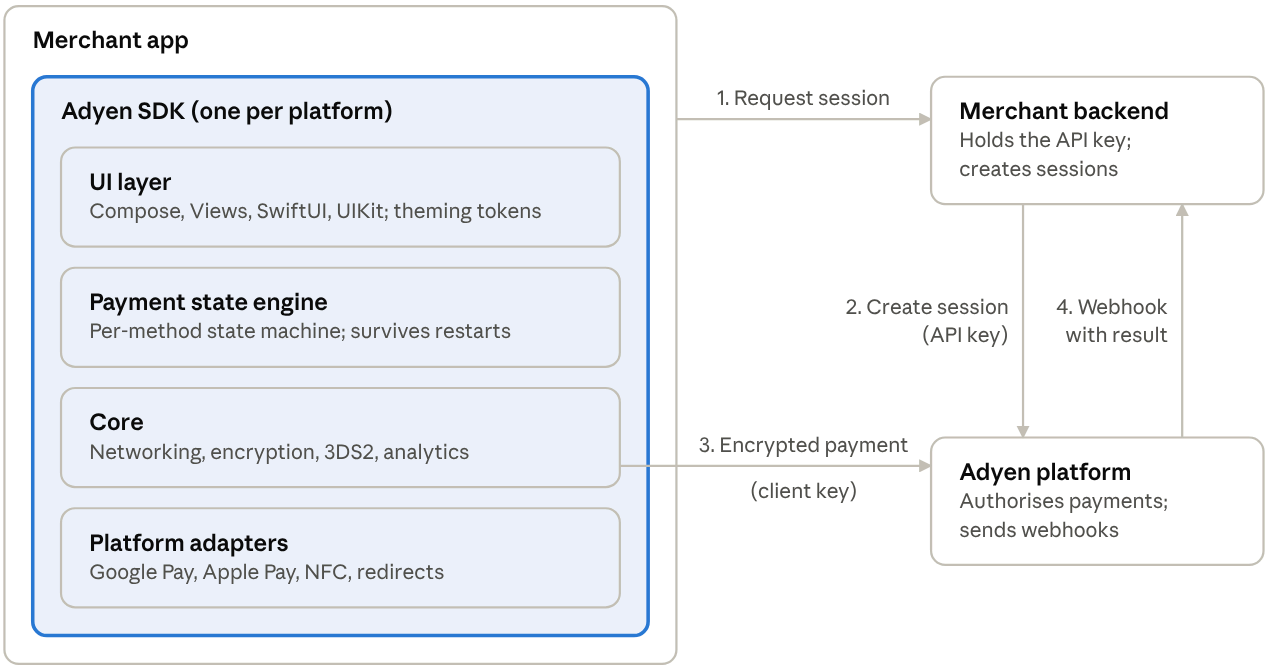
Practice prompts with answer outlines

1. Design a mobile checkout SDK for Android and iOS (cards, wallets, redirect methods). The most likely prompt; rehearse it until it takes 45 minutes.

- Clarify: Drop-in (full UI) or Components (per method) or both; which regions; who owns the backend session.
- API: `Checkout.start(session, config) → result callback` on both platforms; the merchant backend creates the session.

- Architecture: UI layer (Compose/Views, SwiftUI/UIKit) → state engine per payment method → core (networking, encryption, analytics, 3DS) → platform adapters (Google Pay, Apple Pay, browser redirect).
- Deep dives: state survival across redirect and process death; client-side encryption; idempotency; modular binary size.
- Evolution: how you add a new payment method without a major version.

Card data goes from the SDK to Adyen; secrets stay on the backend



checkout SDK architecture · 4 layers, 4-step session flow

Draw this on the whiteboard for prompt 1: the merchant backend holds the secret, the SDK encrypts card data before it leaves the device, and the webhook, not the app, is the source of truth for the final result.

2. Design an SDK that turns a merchant's phone into a payment terminal (Tap to Pay).

- Clarify: platform requirements (Apple entitlement, Android NFC and attestation), certification needs, which card schemes.
- Key points: secure PIN entry, device attestation, the card reader running isolated from merchant code, onboarding flow for store staff, offline policy.
- Trade-off to surface: a standalone app the merchant launches vs an SDK embedded in their app (control and certification vs integration friction).

3. A critical bug shipped in SDK v5.2 and 30% of merchants have it. Design the response and the system that prevents the next one.

- Immediate: server-side mitigation or kill switch, patch release, targeted merchant communication via account managers.
- Systemic: canary merchants on release candidates, contract tests against the API, telemetry alarms per SDK version, blameless postmortem.

4. A shopper starts a purchase on a phone and finishes on an in-store terminal. Design the unified journey.

- Key points: a shared payment session or identifier across channels, a handoff via QR or account, idempotent completion, a single result reported to the merchant, and who owns the session state.
- Staff move: draw the boundaries and APIs between the mobile, terminal, and backend teams so each can build independently.

5. Your SDK must support Flutter and React Native merchants. What do you build?

- Thin wrappers over the native SDKs, generated bindings where possible, one source of truth for behavior, and a compatibility matrix. Push back on reimplementing payments logic in Dart or JavaScript.

Stage 2: Integration project

The integration project rewards the candidate who turns an over-ambitious brief into a plan the business, the teams, and the risk owners can all sign. The brief will almost always contain traps: an unrealistic date, conflicting asks, a promise sales already made, and a security or compliance constraint hiding in a sentence. Finding them is the test.

The response template

Use this structure whether you present on a whiteboard, in a doc, or verbally.

1. **Restate the business outcome in one sentence.** Not the feature list; the reason. "Win holiday volume from three enterprise merchants with one integration."
2. **Ask clarifying questions.** Who decided the date and why? What does success look like in numbers? What is fixed (date, scope, people) and what can move?
3. **Evaluate.** List assumptions, risks, dependencies, and conflicts between asks. Rate each risk by likelihood and impact.

4. **Push back with options, not a no.** Give two or three options with what each costs and delivers. Recommend one. Offer a POC to settle the riskiest unknown.
5. **Plan.** Phases, milestones, gates, owners. Size with ranges, not single numbers.
6. **Align and communicate.** Who needs what, how often, in what format (plan below).
7. **Define success and quality.** Business metrics, engineering quality bars, and what "done" means for a pilot vs general availability.
8. **Close with the decision you need.** "I need a yes on option B by Friday to hold the pilot date."

Sample brief to practice on (about two pages in the real thing)

Read it, set a 45-minute timer, and write your response before reading the worked answer.

> **Project Nova: one SDK for in-app and in-person payments**
>
> Three enterprise pilot merchants (a national retailer, a food delivery app, an event ticketing company) want to accept in-app payments and in-person Tap to Pay through a single integration. Sales has committed to a pilot in 12 weeks, before peak season.
>
> **Product asks:** Android and iOS on day one. One API for both in-app and in-person. Offline payments for venues with poor connectivity (sales mentioned this to the ticketing merchant). A Flutter version for the delivery app. Fully customizable UI so merchants can match their brand, including the card entry form.
>
> **Security asks:** Certificate pinning on all SDK traffic.
>
> **Teams:** Android in Amsterdam (6 engineers), iOS in Chicago (5), Terminal in Madrid (8), Backend in São Paulo (7). No new headcount. Each team has its own roadmap commitments this quarter.
>
> **Legal note:** EU merchants must meet strong customer authentication rules.

Worked answer: what a Staff engineer spots

Ask	The trap	Pushback and alternative
Android + iOS + Tap to Pay in 12 weeks	Four teams, existing commitments, no headcount; iOS Tap to Pay needs platform entitlements and certification you do not control	Pilot Android in-app + Tap to Pay with one merchant; iOS in-app only for pilot; iOS Tap to Pay in phase 2 once entitlement timing is known
One API for in-app and in-person	Sounds simple, but the flows differ (shopper's device vs staff device, card-present rules)	One session and result model, two entry points. Shared core, separate surfaces
Offline payments	A risk and liability decision, not a feature; sales promised it without Risk	Out of pilot scope. Bring Risk, Legal, and the merchant together; define limits; decide with data from the pilot
Flutter version	A second platform to support forever	Thin Flutter wrapper over native SDKs after the native API is stable (phase 2)
Fully customizable card form	Arbitrary UI in card fields breaks PCI scope and secure input	Theming tokens and layout options; secure fields stay SDK-owned
Certificate pinning	A key rotation could break every merchant app in the field	Pin to backup keys with a remote kill switch, or rely on TLS plus certificate transparency; decide with Security in an RFC
Strong customer authentication	Hidden requirement that changes the flow	3DS2 in scope from day one for EU; Legal review gate before pilot

Options to present:

- **Option A: everything in 12 weeks.** High risk of a broken pilot at peak season, burned-out teams, and missed existing commitments. Not recommended.

- **Option B (recommended): phased.** Pilot one merchant on Android (in-app + Tap to Pay) and iOS in-app in 12 weeks; expand in phase 2. Protects the date and the relationship.
- **Option C: delay to after peak.** Lowest risk, but loses the commercial window. Present it so leadership owns the trade-off.

POC to de-risk first (weeks 1–2): a single shared session and result model working end to end on Android across in-app and Tap to Pay. It proves the "one integration" promise early or kills it cheaply.

Cross-functional communication plan

Four sites span about 7 hours (Chicago to Amsterdam and Madrid), so default to written and asynchronous, with one shared live window in the Chicago morning.

Audience	What they need	Format	Cadence
Executives and Sales	Date confidence, risks, decisions needed	One-page status: green/amber/red, top 3 risks, asks	Every two weeks
Product and Design	Scope trade-offs, API and UX decisions	Decision sync plus a written decision log	Weekly
Engineering leads (4 teams)	Interfaces, dependencies, blockers	RFCs and architecture decision records, async review; one live sync in the shared overlap window, recorded	Weekly sync, async daily
Security, Risk, Legal	Early visibility, review gates	Threat model and design review before build, sign-off before pilot	At gates
Pilot merchants	What to expect, integration support	Via account managers; shared integration guide and a test environment	Every two weeks

The Staff behaviors to name: write the RFC that defines the shared API contract so teams build in parallel; keep one decision log so nobody re-litigates; escalate trade-offs early with options attached.

Success metrics

- **Business:** pilot merchant live before peak; share of their volume through the new SDK; integration time for merchant developers (days, not weeks).
- **Quality:** payment success rate equal to or better than the existing integration; crash-free sessions above an agreed bar; zero card data in logs; binary size within budget.
- **Team:** existing roadmap commitments still met; no team blocked more than two days on another.

Stage 3: Chat with the manager

The manager is deciding one thing: would this person make my four teams better without me managing them? Answer first, then the detail, then check if they want more. Show that you understand before you build.

Communication habits that read as Staff

- **Bottom line first.** "Short answer: I'd phase it. Here's why." Then two or three supporting points.
- **Check the depth.** "Want me to go deeper on the sizing or move on?" It shows you manage the listener's time.
- **Ask before solving.** When they describe a problem, ask one or two questions first: "Who feels this pain most today? What has been tried?"
- **Numbers over adjectives.** "Cut integration time from three weeks to four days" beats "made integration much easier."
- **"We" for the team, "I" for your decisions.** They need to hear what you personally decided, wrote, and owned.
- **Be open on the spot.** If they challenge an idea, engage: "That's a fair point; it changes my answer if X is true." Changing your mind with reasons is a Staff signal.

Draw your work as a landscape

When they say "tell me about your work," offer to sketch it. Four layers, top to bottom, with arrows showing value flowing up:

1. **Customers and outcome.** Who uses it and what changed for them, with a number.
2. **Products and surfaces.** The SDK, the sample app, the docs, the API.

3. **Systems you designed.** Modules, connectivity, data flow; circle what you personally built.
4. **People.** Teams and partners you worked with, people you mentored, decisions you drove.

Practice it with your dive-computer SDK: dive computers and divers at the top; the Kotlin SDK and its sample app in the middle; device connection, protocol parsing, log download, and storage below; then who you worked with. Then draw the bridge for them: connecting to an external device, handling flaky connections, and designing an API other developers adopt are the same problems as payment terminals and Tap to Pay.

Sizing work

Walk through it as a method, not a guess.

1. **Break it down** until each piece is a few days to two weeks.
2. **Compare to past work** ("the last payment method we added took six weeks").
3. **Give ranges with confidence:** "8 to 12 weeks, 70% confident; the swing is platform certification."
4. **Add the hidden work:** integration, security review, certification, documentation, merchant support, release.
5. **Name what would shrink it:** cutting a platform, a payment method, or customization.
6. **Re-estimate at gates**, after the POC and after the first integration.

Scaling software and teams

- **Software:** clear module boundaries, a stable public API, contract tests between teams, and paved roads (templates, CI, release tooling) so new work starts consistent.
- **Teams:** ownership maps, an RFC process, decision records, and growing other people into owners so you are never the bottleneck.
- **The line to use:** "My job is to make the right thing the easy thing for four teams."

Time management

Describe how you split your week deliberately. A reasonable example to adapt: about a third hands-on (code, POCs, critical reviews), a quarter design and RFC review, a fifth mentoring, the rest cross-functional work with Product, Design, Security, and partners. Mention protecting deep-work blocks and using time-zone overlap only for decisions that need it.

Questions to expect, with answer skeletons

Question	Skeleton
Tell me about a time you pushed back	The ask, the risk you saw, the data or POC you used, the alternative you offered, the outcome and what you learned
How do you decide what to work on?	Business impact, risk reduction, unblocking others; what you said no to and how
A senior engineer disagrees with your design	Ask for their reasoning, find the real constraint, prototype both if cheap, decide and write it down, commit together
How would you spend your first 90 days?	30: listen, read code and incidents, meet each team and partner. 60: one quick win, map pain points, first RFC. 90: a technical strategy for the year, agreed with the manager
How do you handle a missed deadline?	Early signal, options with costs, re-plan, communicate to stakeholders, fix the cause
Why Adyen, why this role?	Tie your SDK and device work to their mobile and terminal direction; one specific product you studied

Build your story bank

Prepare one story per row before the interviews. Fill each with a situation, your action, a number, and a lesson. Keep each to two minutes spoken.

- [ ] Designing an API other developers adopted (start from the dive-computer SDK)
- [ ] Pushing back on scope or a date, with data
- [ ] A production incident you led or learned from
- [ ] Mentoring someone to a promotion or to owning a system
- [ ] Learning a new platform fast (Views to Compose)
- [ ] Aligning people who disagreed across teams
- [ ] A trade-off you got wrong and how you corrected it

Mentorship: learn and teach

Mentorship is scored in every round, not just when asked, so show it inside your design and project answers too. The Adyen description asks for something specific: coaching tailored to level.

Teach by level

Level	What they need from you	How you do it
Junior	The why behind decisions; how the system fits together	Pairing, annotated code reviews, a system map, explaining one past decision per week
Mid-level	Ownership of a well-defined piece	Give a scoped problem with clear success criteria, review their design before code
Senior	Independence on ambiguous, cross-team problems	Ask questions instead of answering; let them run the RFC; sponsor them in rooms they are not in

Habits to name in interviews

- **Design reviews as teaching.** Ask "what would make this fail?" rather than pointing at the flaw.
- **Delegate the interesting work.** Keep yourself on the riskiest unknown and the glue; hand off the rest with support.
- **Write things down.** Decision records and guides scale mentoring beyond the people you can meet.
- **Measure it.** People who now own systems, reviews they lead without you, promotions you supported.

Show that you learn too

The role needs someone who can cross-train onto iOS. Turn your gap into a strength with a concrete plan:

1. **Weeks 1–4:** Swift, SwiftUI, async/await, and Swift Package Manager by porting a small module of your dive-computer SDK to iOS.

2. **Weeks 5–8:** pair with an iOS senior on real tickets; review iOS pull requests to learn the codebase's patterns.
3. **Ongoing:** trade knowledge. You teach Compose and Kotlin patterns to iOS engineers; they teach you UIKit and Apple platform rules. That builds the cross-platform culture the role asks for.

Your Views-to-Compose move is a ready story: how you learned it, what surprised you, and how you would help a team migrate without a rewrite (interop, screen-by-screen, shared state layer).

Staff behaviors across all stages

- Clarify the problem before solving it.
- State assumptions, trade-offs, and decisions out loud.
- Push back with options and data, never a flat no.
- Name what you would not build.
- Think about the merchant developer and the shopper, not only the code.
- Make other people and teams faster; that is the job.

Study plan, glossary, and questions to ask

A two-week plan at about 90 minutes a day covers all three stages; stretch or compress it to your interview dates.

Two-week plan

- [ ] **Days 1–2:** Read Adyen's Android and iOS SDK repos and online payments docs. Write down how Drop-in, Components, Sessions, and 3DS fit together.
- [ ] **Days 3–4:** Whiteboard practice prompt 1 (checkout SDK) twice, timed at 45 minutes. Record yourself.
- [ ] **Day 5:** Security and payments correctness: rehearse the security and reliability lists out loud.
- [ ] **Day 6:** Practice prompts 2 and 4 (Tap to Pay, unified journey).
- [ ] **Day 7:** CI/CD and release: draw your ideal SDK pipeline from pull request to Maven Central.
- [ ] **Day 8:** Integration project: write a response to the Nova brief in 45 minutes, then compare with the worked answer.

- [ ] **Day 9:** Write your own brief with five traps and solve it; or swap briefs with a friend.
- [ ] **Day 10:** Draw your work landscape; rehearse the dive-computer SDK story in 2 minutes and 5 minutes.
- [ ] **Day 11:** Fill the story bank; time each story.
- [ ] **Day 12:** Mock manager chat with a friend: sizing, 90-day plan, pushback story.
- [ ] **Day 13:** Full mock design round with a peer playing a skeptical interviewer.
- [ ] **Day 14:** Light review; prepare questions; rest.

Glossary to speak fluently

Term	Meaning
PAN	The card number; the thing you keep out of merchant hands
PCI DSS	The card industry security standard; SDK design aims to shrink merchant scope
SCA / PSD2	EU rules requiring strong customer authentication for many online payments
3DS2	The protocol that performs that authentication, often with an app or biometric challenge
Client-side encryption	Encrypting card data in the SDK with the processor's public key
Tokenization	Replacing card data with a token for stored or recurring payments
Drop-in vs Components	All-in-one checkout UI vs per-payment-method building blocks
Client key vs API key	Public, scoped key in the app vs secret key on the merchant server
Webhook (HMAC-signed)	The server notification that confirms the final payment outcome
Idempotency key	A unique request ID that prevents a retry from charging twice

Term	Meaning
EMV	The chip and contactless card standard used at terminals
Tap to Pay / SoftPOS	Accepting contactless cards on a phone without extra hardware
RFC / ADR	A proposal for review / a record of a decision and its reasons

Questions to ask them

- What does success look like for this role in the first six months?
- Where do the mobile SDK and terminal teams disagree most today?
- How long do merchants typically stay on an old SDK version, and how do you handle it?
- How are public API changes decided across platforms?
- What share of a Staff engineer's time here is hands-on code?
- What is the hardest technical problem the teams are facing right now?
- How does Adyen grow Staff engineers further?

Before the interviews, read Adyen's published values, the Adyen Formula, and map one story from your bank to each.